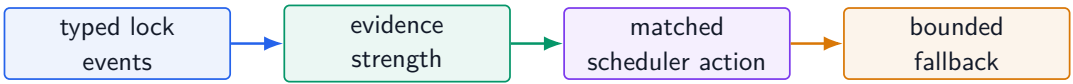
Phase 1

锁观测与 Owner 追踪正确性

让证据强度匹配调度动作，而不是让传感器猜测锁管理器状态



private futex wait/wake、kernel mutex、shared/PI/requeue 分开解释

完成依据：三 scheduler build、static/BTF smoke、controlled runtime、MariaDB coverage-only 与单 case 回归。

一句话结论：Phase 1 修的是证据语义

20.06%	30.01%	0	0
MariaDB private-owner resolver coverage	MariaDB locality decision hit	resolver-off boost	runtime map update failure

完成：普通 futex wake 不再被写成 owner；LockNest 可消费 advisory anchor，LockBoost 只消费 verified owner。

边界：owner hit 是证据覆盖率，不是应用侧 owner accuracy；当前 resolver 只对应已验证的 x86-64 glibc/NPTL profile。

关键结果：resolver-off controlled run 产生 21,339 private futex wait、21,026 locality hit、0 boost；resolver-on 产生 19,661 owner hit、937 boost。

审稿人真正质疑的核心：观测到的到底是什么？

容易被误读的强假设

- `FUTEX_WAKE` 的调用者就是 owner。
- kernel mutex 与 pthread mutex 是同一个内核路径。
- 一个 lock address 可以跨进程全局标识锁。
- owner 不确定时仍然应该继续 boost。

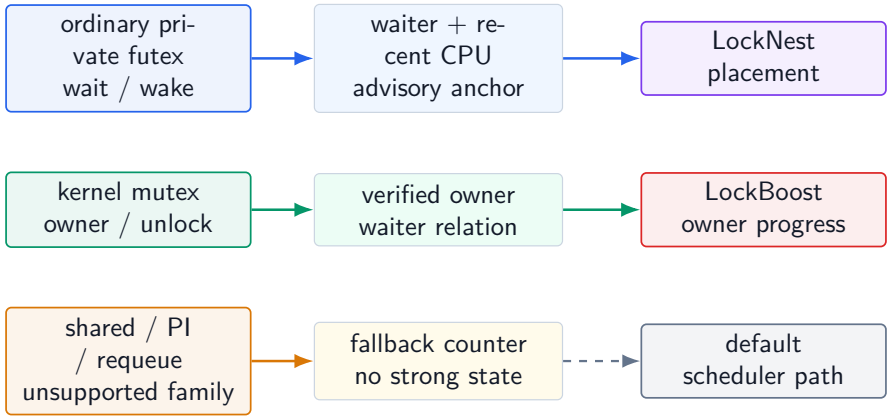
错误 owner + 强动作 = 把观测误差放大成调度干扰。

Phase 1 的回答

- 事件按语义分类，而不是统一称为 lock event。
- owner 证据单独建模，并要求 TID / 地址空间检查。
- LockNest 与 LockBoost 使用不同证据门控。
- 证据缺失只关闭强动作，保留可安全回退的 locality hint。

核心原则：**evidence strength 决定 action strength**。

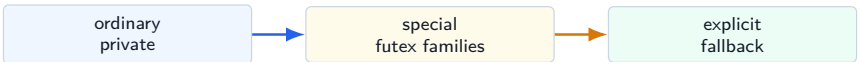
证据到动作的契约：三条路径，三种语义



同一事件可以被两个调度器不同地消费：wake 能帮助 LockNest 选择位置，但不能单独证明 LockBoost 的 owner。

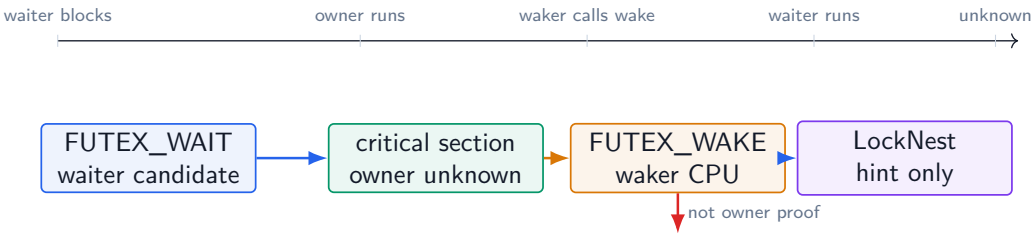
Lock event taxonomy：支持什么，主动不支持什么

事件族	观测含义	LockNest	LockBoost	当前 claim
kernel mutex acquire/unlock	task 与内核 mutex 的直接关系	strong anchor	verified owner	supported
private futex wait	waiter 可能阻塞在 uaddr	waiter candidate	resolver 可选	profile-dependent
private futex wake	调用者执行唤醒	recent CPU hint	不写 owner map	advisory only
shared futex	key 需要 mapping identity	fallback	fallback	unsupported
PI futex	owner/优先级继承语义不同	fallback	fallback	unsupported
requeue / wake-op	waiter 在 futex 间迁移	fallback	fallback	unsupported



“不支持”不是遗漏：这些路径被分类、计数并退化，避免在 fast path 中伪造错误 owner 关系。

Futex wait/wake 语义：一个 wake 事件不能回答 owner 是谁



修复后的路径：wake 更新 `futex_hints`，不写 `lock_owners`，不创建 waiter-owner dependency。

代码修复 1：从 waker-as-owner 到 evidence split

旧路径：错误的强推断

```
FUTEX_WAKE_RETURN{
  owner = current;
  lock_owners[key] = owner;
  add_waiter(owner);
  maybe_boost(owner);
}
```

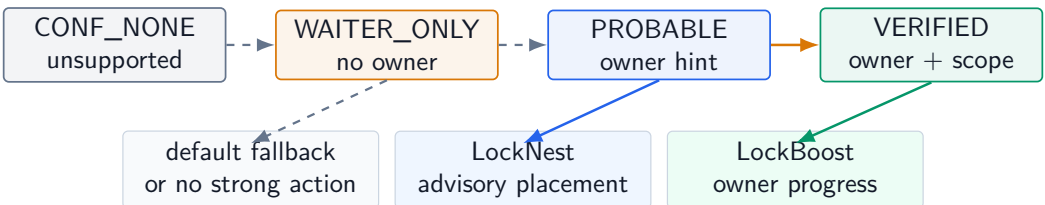
waker、owner 和 waiter 被错误地压缩成同一个角色。

当前路径：两条证据链

```
FUTEX_WAKE_RETURN{
  futex_hints[key] = waker_cpu;
  refresh_anchor(key);
  // no owner map write
}
OWNER_RESOLVED{
  if (verified) add_waiter(owner);
}
```

弱证据继续服务 LockNest；强动作只由 verified owner 触发。

Owner evidence ladder：强动作必须匹配强证据



Phase 1 的策略不是让所有事件进入最高等级，而是让低置信事件安全退化。

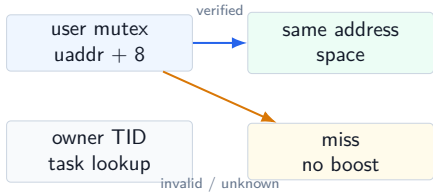
glibc/NPTL resolver：offset=8 是部署 profile，不是通用 futex 语义

解析与验证条件

$$owner_tid = *(u32*)(uaddr + 8)$$

$$verified \iff tid \neq 0 \wedge task(tid) \neq \varnothing \wedge mm(tid) = mm(waiter)$$

- 本机 x86-64 glibc 2.39/NPTL 验证 `__owner` offset 为 8。
- `-O -1` 关闭 resolver，回到保守路径。
- hit/miss 表示 resolver evidence coverage，不表示 accuracy。



其它 libc、ABI、架构和 lock family 需要重新验证，不能直接复用 offset=8。

数据结构：把“谁在等谁”拆成可审计状态

evidence maps

`lock_owners[key]`
verified owner / CPU / source

`futex_hints[key]`
waker CPU / observed time

`waiters[tid]`
key / anchor / optional owner

action state

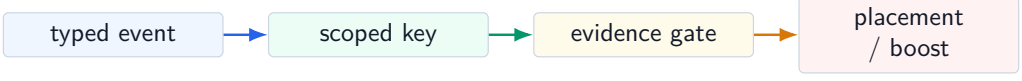
`contended_tasks[owner]`
waiter count / owner valid

`active_mutex_lock[tid]`
enter / return pairing

decay / cleanup markers

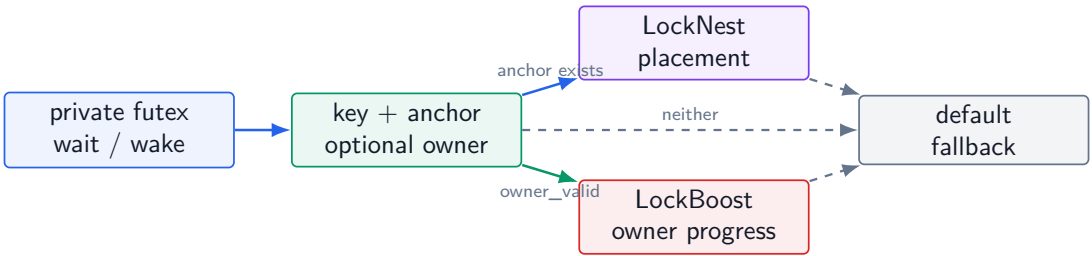
diagnostic state

per-CPU stats array
fallback families
map failures
underflow / overwrite
-D only for evidence mode



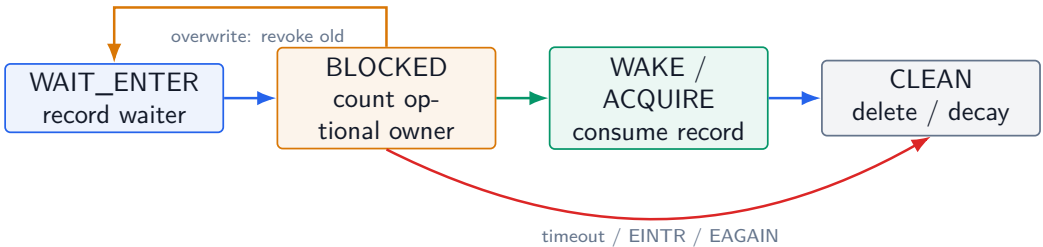
关键设计：owner-valid 是 waiter state 的显式字段；没有 owner 时，waiter 仍然可以服务 LockNest，但不能增加 LockBoost contention。

组合调度器：LockNest 与 LockBoost 正交消费证据



有 anchor、无 owner：只 placement；有 owner、无 anchor：只 owner progress；两者都没有：默认调度。

Waiter 生命周期：异常返回必须能回到无状态



清理触发点：sys_exit_futex、mutex acquire、select_cpu、decay。

一个 TID 只保留一个 active blocking dependency，覆盖前先撤销旧 owner count。

代码修复 2：从 verifier rejection 到固定上界 CAS

旧实现：无界重试

```
for (;;) {
  old = *count;
  if (CAS(count, old, old-1))
    break;
}
```

真实 attach 暴露 BPF verifier rejection；不是理论担忧，而是不可加载的实现缺陷。

当前实现：verifier-friendly bounded CAS

```
#pragma unroll
for (i = 0; i < 4; i++) {
  if (CAS(count, old, old-1))
    return;
}
counter_underflow++;
return;
```

固定四次重试通过 build / BTF / live attach；失败时只丢失一次 bookkeeping 动作。

bounded state update $\Longrightarrow$ no unbounded verifier path + no permanent boost

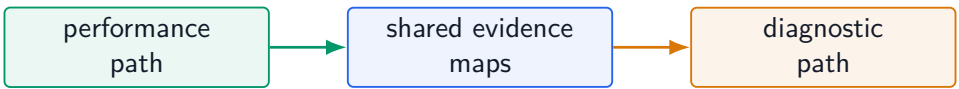
性能保护：诊断路径与正式性能路径分离

正式性能模式

- 不加载 futex exit / sched wake diagnostic probes。
- BPF map lookup 保持在必要的调度路径内。
- 只保留 LockNest placement 与 LockBoost action。
- 单 case regression 只作为不劣化 gate。

机制证据模式：`-D`

- 打开完整 counters 与 fallback family 统计。
- 记录 owner hit/miss、stale、underflow、map failure。
- 用 coverage-only 解释证据边界。
- 不把诊断结果当作 TPS 排名。



Controlled runtime：验证路径和不变量，而不是排名

覆盖的同步路径

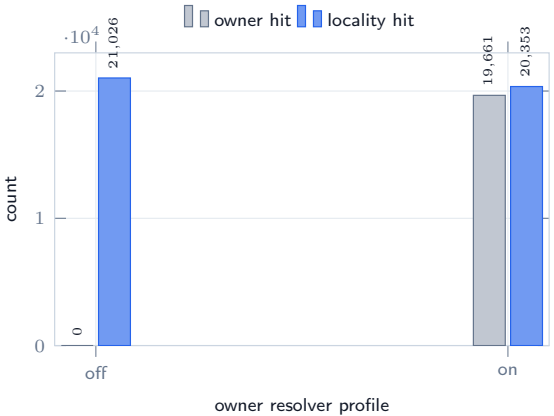
case	purpose
private pthread mutex	resolver / waiter / boost
same-address alias	scoped identity
address reuse	stale advisory state
shared futex	explicit fallback
PI futex	unsupported fallback
requeue / wake-only	semantic boundary
1,152 locks	map pressure

每次 run 的验收

- scheduler return code = 0
- benchmark return code = 0
- no `runnable task stall`
- no `scx_bpf_error`
- sched_ext after run = disabled
- key scope / fallback counters 可解析

短 controlled run 负责回答 “路径是否正确”；不把短跑误写成 owner accuracy 或长期公平性。

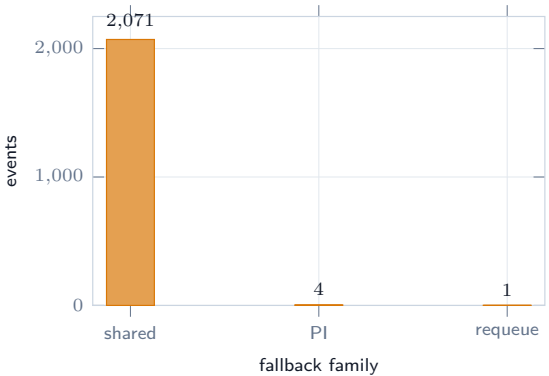
Resolver-off / resolver-on：证据门控改变了动作，而非路径安全



metric	off	on
private wait	21,339	20,877
owner hit	0	19,661
owner miss	21,339	1,216
boost	0	937
stale nest	4	2
map failure	0	0

关闭 resolver 时 owner hit/boost 为 0，但 locality 仍然工作。

Fallback family：不支持路径被计数，而不是被伪装



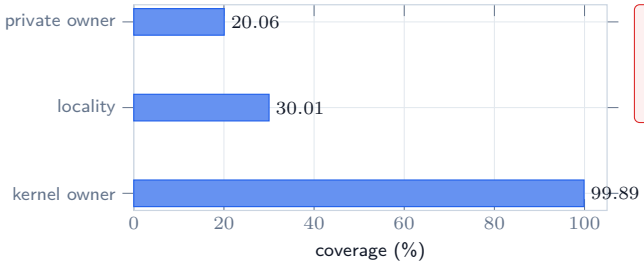
resolver-off controlled run

family	action
shared futex	fallback only
PI futex	fallback only
requeue	fallback only

当前负载中的 PI/requeue 次数很低；因此论文只能报告 boundary，不可宣称通用支持。

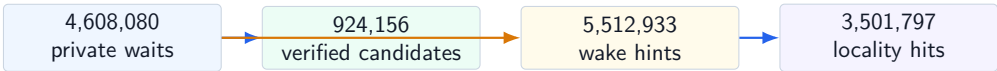
MariaDB coverage-only：三种 coverage 必须分开报告

coverage dimension	numerator	denominator	result
private owner resolver	924,156	924,156 + 3,683,927	20.06% evidence coverage
LockNest locality decision	3,501,797	3,501,797 + 8,166,080	30.01% placement hit
kernel mutex owner	9,437	9,437 + 10	99.89% kernel-path hit
ordinary private futex wake	5,512,933	–	advisory hint only
shared / PI / requeue	17 / 0 / 0	–	explicit fallback



不能把 `owner resolver hit` 写成 owner accuracy；没有应用侧 ground truth 时，论文 claim 必须停在 selective verified-owner boosting。

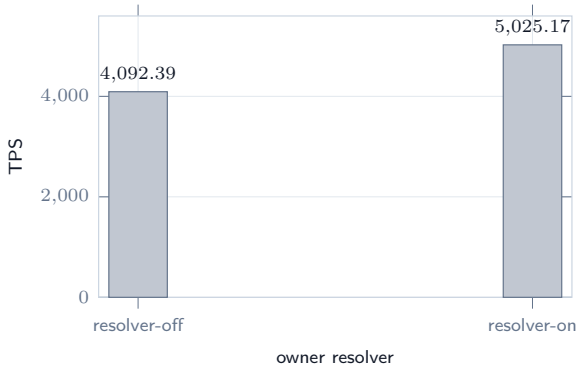
Mechanism evidence：owner coverage 低并不等于 LockNest 失效



两条证据链并行：owner resolver **只服务** LockBoost；wake hint **即使不能证明** owner，**仍可能刷新** LockNest anchor。

结果口径：coverage-only 统计只回答“哪些路径可被当前实现识别”，不回答应用锁管理器的完整语义。

单 case 性能回归：正确性修复没有破坏当前 MariaDB 主线



profile	TPS	p99
resolver-off	4092.39	–
resolver-on	**5025.17**	**137.35 ms**

	p95	scheduler
resolver-on	102.97 ms	no stall
both runs	–	disabled after run

该单 case 只作为 Phase 1/2 no-regression gate，不替代正式性能矩阵。

状态安全 counters：保护分支出现，但没有演化成调度器失败

counter	resolver-off	resolver-on	MariaDB pooled
waiter overwrite	0	0	26
counter underflow guard	0	0	39
map update failure	0	0	0
stale nest	4	2	88,474
waiter cleanup	21,261	20,734	4,544,084
decay cleanup	0	125	40,713

map update failure=0，所有 controlled run scheduler/bench 返回 0，sched_ext 最终回到 disabled。

underflow guard 非零说明保护分支确实被需要；应写成“有界防护”，不能写成形式化无竞态。

代码优化过程：从可加载、可解释到可回退

问题	初始策略	当前策略	结果
wake/owner 混淆	waker 写 owner map	wake 只写 futex hint	owner 语义分离
kretprobe 误读	读取不存在的返回值	return probe 只确认当前 task	path 可加载
resolver 过强	猜任意 futex owner	offset=8 + TID + mm gate	selective claim
count 更新	无界 CAS loop	固定 4 次 CAS + guard	verifier 通过
异常退出	waiter 长期残留	sys_exit/acquire/select_cpu/decay 清理	bounded state
诊断开销	所有 probe 常开	性能模式关闭 diagnostic-only path	no-regression gate

优化方向不是堆叠判断，而是把不确定性变成廉价、显式、可回退的状态。

Phase 1 完成依据：机制、实验、claim 三层一致

机制

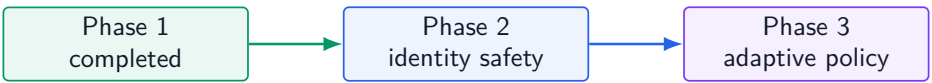
- typed events
- owner gate
- waiter cleanup
- bounded CAS

实验

- off/on controlled
- fallback families
- alias/reuse paths
- DB coverage-only

claim

- advisory locality
- verified owner
- no accuracy claim
- explicit unsupported paths



Phase 1 已完成其自身目标：不再让 LockBoost 建立在普通 futex wake 的错误 owner 假设上。

Phase 1 的论文 claim 边界

可以写

- ordinary private futex 可提供 waiter / advisory anchor evidence；
- LockBoost 只对 kernel mutex 或 validated glibc/NPTL owner 进行 boosting；
- resolver-off 时系统安全退化且 LockNest 仍可工作；
- shared / PI / requeue 被显式分类并 fallback。

不能写

- 完整追踪所有用户态 futex owner；
- owner hit rate 等于 owner accuracy；
- PI/requeue 已经被支持；
- 不增加任何用户态 ABI / libc profile 假设。

推荐论文表述：selective verified-owner boosting plus advisory lock-locality placement。

Phase 1：evidence before action

private futex hint、verified owner、fallback boundary

